Logique du premier ordre

Syntaxe, sémantique, déduction naturelle, résolution

Emmanuel Filiot et Jean-François Raskin

Université libre de Bruxelles

Plan

1. Syntaxe
2. Sémantique
3. Déduction naturelle
4. Résolution en logique propositionnelle
5. Résolution au premier ordre : formes normales
6. Unification
7. Résolution au premier ordre

Plan

1. Syntaxe
2. Sémantique
3. Déduction naturelle
4. Résolution en logique propositionnelle
5. Résolution au premier ordre : formes normales
6. Unification
7. Résolution au premier ordre

Pourquoi le premier ordre ?

Par rapport à la logique propositionnelle, la logique du premier ordre (ou logique des prédicats) apporte :

- des variables qui prennent leurs valeurs dans un domaine quelconque, et plus seulement dans $\{0, 1\}$;
- des quantificateurs $\forall$ et $\exists$;
- des symboles de relation (prédicats), qui décrivent des relations entre valeurs ;
- des symboles de fonction, qui permettent de construire des termes.

On parle de « premier ordre » parce qu'on quantifie uniquement sur les éléments du domaine. Une logique *d'ordre supérieur* permettrait de quantifier aussi sur les fonctions et les relations.

Un exemple : la conjecture de Goldbach

« Tout entier pair strictement plus grand que 2 est somme de deux nombres premiers. »

$$\forall x \left(\mathit{pair}(x) \wedge 2 < x \;\rightarrow\; \exists y \, \exists z \left(\mathit{premier}(y) \wedge \mathit{premier}(z) \wedge x = y + z \right) \right)$$

- *pair*, *premier*, $<$ et $=$ sont des *prédicats* ;
- $+$ est un *symbole de fonction* et 2 une *constante* ;
- $y + z$ est un *terme* : il désigne un élément du domaine.

Cette phrase parle d'une infinité d'entiers : elle est inexprimable en logique propositionnelle.

Langages

Symboles communs à *tous* les langages : les connecteurs $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$, les quantificateurs $\forall, \exists$, les parenthèses et un ensemble infini V de variables $x, y, z, \ldots$

Définition 1 (Langage)

Un langage $\mathcal{L}$ est déterminé par ses symboles propres :

- des *prédicats* $p, q, r, \ldots$, chacun d'arité $n \geq 1$ (on écrit $p|_n$) ;
- des *symboles de fonction* $f, g, h, \ldots$, chacun d'arité $n \geq 1$ ($f|_n$) ;
- des *constantes* $c, d, \ldots$

Le prédicat d'égalité $=$ est toujours disponible.

Exemple 2

$\mathcal{L}_{ar} = \{0,\ s|_1,\ +|_2,\ <|_2\}$: une constante, une fonction « successeur » unaire, une fonction binaire et un prédicat binaire.

Termes

Définition 3 (Termes)

L'ensemble des termes est le plus petit ensemble qui

1. contient les constantes et les variables ;
2. contient $f(t_1, \ldots, t_n)$ dès que f est d'arité n et $t_1, \ldots, t_n$ sont des termes.

Un terme sans variable est dit clos.

Exemple 4 (dans $\mathcal{L}_{ar} = \{0, s|_1, +|_2, <|_2\}$)

- 0, $s(s(0))$, $x + s(0)$ et $s(x + y)$ sont des termes ; $s(s(0))$ est clos.
- $x < s(x)$ n'est *pas* un terme, car $<$ est un prédicat : c'est une formule atomique.
- $s(0, x)$ n'est rien du tout, car s est d'arité 1.
- $s(x < y)$ n'est pas bien formé : une fonction s'applique à des termes, jamais à une formule.

Formules

Définition 5 (Formules)

Une formule atomique est de la forme $p(t_1, \ldots, t_n)$, avec p d'arité n et $t_1, \ldots, t_n$ des termes. On écrit $t_1 = t_2$ plutôt que $= (t_1, t_2)$.
Les formules sont engendrées par la grammaire

$$\phi ::= p(t_1, \ldots, t_n) \mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \to \phi \mid \phi \leftrightarrow \phi \mid \exists x \cdot \phi \mid \forall x \cdot \phi$$

Les parenthèses ne sont que de la ponctuation, pour lever les ambiguïtés.

Priorités

Les connecteurs gardent leurs priorités propositionnelles et les quantificateurs ont la *même priorité que* $\neg$: $\forall x \cdot \neg p(x, y) \vee p(y, x)$ se lit $(\forall x \cdot \neg p(x, y)) \vee p(y, x)$.

Exemple 6 (piège)

$\exists x\, p(x) \wedge q(x)$ se lit $(\exists x\, p(x)) \wedge q(x)$: le x de $q(x)$ n'est *pas* lié. Pour qu'un même élément vérifie p et q, il faut écrire $\exists x\, (p(x) \wedge q(x))$.

Traduire le langage naturel

Avec un prédicat binaire *aime* et une fonction unaire f :

- « Tout le monde aime quelqu'un » : $\forall x \, \exists y \, \mathit{aime}(x, y)$
- « Quelqu'un est aimé de tous » : $\exists y \, \forall x \, \mathit{aime}(x, y)$
  $\rightsquigarrow$ seul l'ordre des quantificateurs change, et le sens est différent.
- « f est injective » : $\forall x \forall y \, (f(x) = f(y) \rightarrow x = y)$
- « f est surjective » : $\forall y \, \exists x \, f(x) = y$
- « Au moins deux éléments » : $\exists x \exists y \, \neg(x = y)$
- « Au plus un élément » : $\forall x \forall y \, x = y$
- « Le prédicat binaire p est le graphe d'une fonction » :

$$\forall x \, \exists y \, p(x, y) \;\; \wedge \;\; \forall x \forall y \forall z \, (p(x, y) \wedge p(x, z) \rightarrow y = z)$$

Variables libres et liées

Une occurrence d'une variable est une position où elle apparaît effectivement, sans compter celle qui suit immédiatement un quantificateur.

Définition 7

- Une occurrence de x dans ϕ est liée si elle se trouve dans une sous-formule qui commence par $\forall x$ ou $\exists x$. Sinon, elle est libre.
- x est *libre dans* ϕ si elle y a au moins une occurrence libre. On note $\mathit{Libres}(\phi)$ l'ensemble de ces variables.
- ϕ est close si $\mathit{Libres}(\phi) = \varnothing$.
- On écrit $\phi(x_1, \ldots, x_n)$ lorsque $\mathit{Libres}(\phi) = \{x_1, \ldots, x_n\}$.

Exemple 8

Soit $\phi \equiv \big(\forall x\,(p(x) \vee \exists y\, r(x,y))\big) \to q(y)$.

- x a deux occurrences, toutes deux liées par $\forall x$.
- y a deux occurrences : celle de $r(x,y)$ est liée par $\exists y$, celle de $q(y)$ est libre, car la portée de $\exists y$ se limite à $r(x,y)$.

Donc $\mathit{Libres}(\phi) = \{y\}$: une même variable peut être à la fois libre et liée.

Substitution et capture de variables

Définition 9

- $\phi[t/x]$ est la formule obtenue en remplaçant *toutes les occurrences libres* de x par le terme t.
- t est libre pour x dans ϕ si aucune occurrence libre de x ne se trouve sous un $\forall y$ ou un $\exists y$ portant sur une variable y de t.

Autrement dit, la substitution ne doit *capturer* aucune variable de t.

Exemple 10

Sur $(\mathbb{N}, \leq)$, $\phi(x) \equiv \forall y\,(x \leq y)$ signifie « x est le plus petit élément » et n'est vraie que pour $x = 0$.

- $\phi[z/x] \equiv \forall y\,(z \leq y)$ garde le même sens : z est libre pour x.
- $\phi[s(z)/x] \equiv \forall y\,(s(z) \leq y)$ est correcte aussi.
- y n'est pas libre pour x : on obtiendrait $\forall y\,(y \leq y)$, qui est toujours vraie. Il faut d'abord renommer en $\forall y'\,(x \leq y')$, puis substituer.

Plan

1. Syntaxe
2. Sémantique
3. Déduction naturelle
4. Résolution en logique propositionnelle
5. Résolution au premier ordre : formes normales
6. Unification
7. Résolution au premier ordre

Structures

Définition 11 (Structure)

Une structure $\mathcal{M}$ pour $\mathcal{L}$ est la donnée

- d'un *domaine* $M \neq \varnothing$;
- d'une relation $r^{\mathcal{M}} \subseteq M^n$ pour chaque prédicat $r|_n$;
- d'une fonction totale $f^{\mathcal{M}} : M^m \to M$ pour chaque fonction $f|_m$;
- d'un élément $c^{\mathcal{M}} \in M$ pour chaque constante c.

On l'écrit souvent comme un n-uplet, par exemple $\mathcal{M}_{ar} = (\mathbb{N}, 0, succ, +, <)$ pour $\mathcal{L}_{ar}$.

Définition 12 (Valuation)

Une valuation est une application $v : V \to M$. On note $v[x \mapsto a]$ la valuation qui coïncide avec v, sauf en x, où elle vaut a.

Valeur d'un terme

Définition 13

La valeur $t^{\mathcal{M},v} \in M$ du terme t se définit par induction :

$$c^{\mathcal{M},v} = c^{\mathcal{M}}, \qquad x^{\mathcal{M},v} = v(x), \qquad f(t_1, \ldots, t_n)^{\mathcal{M},v} = f^{\mathcal{M}}(t_1^{\mathcal{M},v}, \ldots, t_n^{\mathcal{M},v}).$$

Exemple 14

Dans $\mathcal{M}_{ar} = (\mathbb{N}, 0, succ, +, <)$, avec $v(x) = 5$ et $v(y) = 2$:

$$\big(s(x) + s(s(0))\big)^{\mathcal{M}_{ar},v} = 6 + 2 = 8, \qquad s(x + y)^{\mathcal{M}_{ar},v} = succ(5 + 2) = 8.$$

Les deux termes sont différents mais ont la même valeur : les termes relèvent de la *syntaxe*, et c'est la structure qui leur donne une valeur.

Satisfaction

Définition 15 ($\mathcal{M}, v \models \phi$)

Par induction sur ϕ :

- $\mathcal{M}, v \models r(t_1, \ldots, t_n)$ ssi $(t_1^{\mathcal{M},v}, \ldots, t_n^{\mathcal{M},v}) \in r^{\mathcal{M}}$;
- $\mathcal{M}, v \models t_1 = t_2$ ssi $t_1^{\mathcal{M},v} = t_2^{\mathcal{M},v}$ (vraie égalité sur M) ;
- pour $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$, on applique les tables de vérité propositionnelles ;
- $\mathcal{M}, v \models \exists x\, \psi$ ssi il existe $a \in M$ tel que $\mathcal{M}, v[x \mapsto a] \models \psi$;
- $\mathcal{M}, v \models \forall x\, \psi$ ssi pour tout $a \in M$, $\mathcal{M}, v[x \mapsto a] \models \psi$.

- On dit alors que $(\mathcal{M}, v)$ est un modèle de ϕ.
- La valeur de ϕ ne dépend que de v restreinte à $\mathit{Libres}(\phi)$. Pour une formule close, elle ne dépend donc pas de v, et l'on écrit $\mathcal{M} \models \phi$.
- Formulation équivalente : on considère les valuations v' qui coïncident avec v sur $\mathit{Libres}(\phi)$.

Le domaine compte, l'ordre des quantificateurs aussi

Exemple 16 (avec $+$ et 0 usuels)

- $\forall x \, \exists y \, (x + y = 0)$:
  - vraie dans $\mathbb{Z}$, en prenant $y = -x$;
  - fausse dans $\mathbb{N}$: pour $x = 1$, aucun $y \in \mathbb{N}$ ne vérifie $1 + y = 0$.
- $\exists y \, \forall x \, (x + y = x)$: vraie dans $\mathbb{N}$ et dans $\mathbb{Z}$, avec $y = 0$.
- Dans $(\mathbb{N}, <)$, $\forall x \, \exists y \, (x < y)$ est vraie, mais $\exists y \, \forall x \, (x < y)$ est fausse.

Exemple 17 (variable libre)

Dans $(\mathbb{N}, <)$, la formule $\phi(x) \equiv \neg \exists y \, (y < x)$ est vraie pour $v(x) = 0$ et fausse pour $v(x) = 3$. Sa valeur dépend de v, puisque x est libre.

Domaines finis

Sur un domaine fini, on peut énumérer l'interprétation d'un prédicat et vérifier une formule à la main.

Exemple 18 (graphe non orienté, $D = \{a, b, c\}$)

$\phi_{graphe} \equiv \forall x \neg p(x, x) \wedge \forall x \forall y \, (p(x, y) \rightarrow p(y, x))$

- $p^{\mathcal{M}} = \{(a, b), (b, a)\}$: modèle ;
- $p^{\mathcal{M}} = \{(a, b), (b, a), (c, c)\}$: pas un modèle, à cause de la boucle en c ;
- $p^{\mathcal{M}} = \{(a, b)\}$: pas un modèle, car la symétrie échoue.

Exemple 19 (ordre partiel, $D = \{1, 2, 3, 6\}$, $p =$ « divise »)

- Modèle de la réflexivité, de la transitivité et de l'antisymétrie $\forall x \forall y \, (p(x, y) \wedge p(y, x) \rightarrow x = y)$.
- Mais $\forall x \forall y \, (p(x, y) \vee p(y, x))$ est fausse, puisque 2 et 3 sont incomparables : c'est un ordre *partiel*.

Satisfaisabilité, validité, équivalence, conséquence

Définition 20

Soient $\phi, \phi_1, \ldots, \phi_n$ des formules closes.

- ϕ est satisfaisable si elle a au moins un modèle ;
- ϕ est valide si toute structure en est un modèle ;
- ϕ est conséquence logique de $\phi_1, \ldots, \phi_n$, noté $\phi_1, \ldots, \phi_n \models \phi$, si tout modèle commun des ϕ_i est un modèle de ϕ.

Deux formules ϕ_1, ϕ_2 qui ont les mêmes variables libres $x_1, \ldots, x_n$ sont équivalentes si $\forall x_1 \ldots \forall x_n (\phi_1 \leftrightarrow \phi_2)$ est valide.

Proposition 21

$\phi_1, \ldots, \phi_n \models \phi$ *ssi* $\phi_1 \wedge \cdots \wedge \phi_n \wedge \neg\phi$ *est insatisfaisable.*

Démonstration.

$\phi_1 \wedge \cdots \wedge \phi_n \wedge \neg\phi$ a un modèle ssi il existe une structure qui satisfait tous les ϕ_i sans satisfaire ϕ, c'est-à-dire ssi la conséquence est fausse. □

C'est sur cette proposition que repose toute la méthode de résolution.

Comment montrer . . .

Méthodes

- . . . qu'une formule n'est pas valide, ou qu'une conséquence ne tient pas : on exhibe *une* structure contre-exemple. Un petit domaine fini comme $\{1, 2\}$ suffit souvent.
- . . . qu'une formule est valide : on raisonne sur une structure et une valuation *quelconques*, ou on en donne une preuve (déduction naturelle, résolution).
- . . . qu'une formule est satisfaisable : on exhibe *un* modèle.

Exemple 22

- $\forall x\, p(x) \to p(c)$ est valide.
- $\exists x\, p(x) \to p(c)$ est satisfaisable mais pas valide : prendre $D = \{1, 2\}$, $p^{\mathcal{M}} = \{1\}$, $c^{\mathcal{M}} = 2$.
- $\forall x\, p(x) \wedge \exists x\, \neg p(x)$ est insatisfaisable.

Conséquences logiques : exemples

Exemple 23

- $\forall x\, p(x) \models \exists x\, p(x)$, puisque le domaine n'est jamais vide.
- $\forall x\, (p(x) \to q(f(x)))$, $p(c) \models \exists y\, q(y)$, avec $f(c)$ comme témoin.
- $\forall x\, \exists y\, r(x, y) \not\models \exists x\, r(x, x)$. Contre-exemple : $(\mathbb{N}, <)$.
- $\exists x\, p(x)$, $\exists x\, q(x) \not\models \exists x\, (p(x) \wedge q(x))$. Contre-exemple : $D = \{1, 2\}$, $p^{\mathcal{M}} = \{1\}$, $q^{\mathcal{M}} = \{2\}$.

Équivalences utiles

Dans la liste ci-dessous, x n'est pas libre dans ψ.

$$\neg\forall x\,\phi \equiv \exists x\,\neg\phi \qquad \neg\exists x\,\phi \equiv \forall x\,\neg\phi$$
$$\forall x\,(\phi \wedge \phi') \equiv \forall x\,\phi \wedge \forall x\,\phi' \qquad \exists x\,(\phi \vee \phi') \equiv \exists x\,\phi \vee \exists x\,\phi'$$
$$(\forall x\,\phi) \vee \psi \equiv \forall x\,(\phi \vee \psi) \qquad (\exists x\,\phi) \wedge \psi \equiv \exists x\,(\phi \wedge \psi)$$
$$(\forall x\,\phi) \wedge \psi \equiv \forall x\,(\phi \wedge \psi) \qquad (\exists x\,\phi) \vee \psi \equiv \exists x\,(\phi \vee \psi)$$
$$\psi \to \forall x\,\phi \equiv \forall x\,(\psi \to \phi) \qquad \psi \to \exists x\,\phi \equiv \exists x\,(\psi \to \phi)$$
$$(\forall x\,\phi) \to \psi \equiv \exists x\,(\phi \to \psi) \qquad (\exists x\,\phi) \to \psi \equiv \forall x\,(\phi \to \psi)$$
$$\forall x\,\phi \equiv \forall y\,\phi[y/x] \qquad \exists x\,\phi \equiv \exists y\,\phi[y/x] \quad (y \text{ absente de } \phi)$$

Ce qui ne marche pas

- $\forall$ ne se distribue pas sur $\vee$, et $\exists$ ne se distribue pas sur $\wedge$.
- $\exists x \forall y\,\phi \models \forall y \exists x\,\phi$, mais la réciproque est fausse.

Un $\exists$ caché ; indécidabilité

Exemple 24

$$\forall x\,(p(x) \to q(c)) \;\equiv\; (\exists x\, p(x)) \to q(c)$$

« Quel que soit x, si x vérifie p, alors $q(c)$ » revient à dire « s'il existe un x qui vérifie p, alors $q(c)$ ».
$\rightsquigarrow$ Un quantificateur qui sort de l'antécédent d'une implication change de nature.

Théorème 25 (Church et Turing, 1936)

Aucun algorithme ne permet de décider si une formule du premier ordre est satisfaisable (ou, de façon équivalente, valide).

Plan

1. Syntaxe
2. Sémantique
3. Déduction naturelle
4. Résolution en logique propositionnelle
5. Résolution au premier ordre : formes normales
6. Unification
7. Résolution au premier ordre

Rappel : règles propositionnelles (1)

Introduction	**Élimination**
$\dfrac{\phi \quad \psi}{\phi \wedge \psi}\ \wedge_i$	$\dfrac{\phi \wedge \psi}{\phi}\ \wedge_{e_1} \qquad \dfrac{\phi \wedge \psi}{\psi}\ \wedge_{e_2}$
$\dfrac{\phi}{\phi \vee \psi}\ \vee_{i_1} \qquad \dfrac{\psi}{\phi \vee \psi}\ \vee_{i_2}$	$\dfrac{\phi \vee \psi \qquad \begin{array}{|c|}\hline \phi \\ \vdots \\ \chi \\ \hline \end{array} \qquad \begin{array}{|c|}\hline \psi \\ \vdots \\ \chi \\ \hline \end{array}}{\chi}\ \vee_e$

Rappel : règles propositionnelles (2)

Introduction	**Élimination**
$\dfrac{\boxed{\begin{array}{c}\phi\\ \vdots\\ \psi\end{array}}}{\phi \to \psi}\ \to_i$	$\dfrac{\phi \qquad \phi \to \psi}{\psi}\ \to_e$
$\dfrac{\boxed{\begin{array}{c}\phi\\ \vdots\\ \bot\end{array}}}{\neg\phi}\ \neg_i$	$\dfrac{\phi \qquad \neg\phi}{\bot}\ \neg_e \qquad \dfrac{\bot}{\phi}\ \bot_e \qquad \dfrac{\neg\neg\phi}{\phi}\ \neg\neg_e$

Règles dérivées et présentation des preuves

Règles dérivées

$$\frac{\phi}{\neg\neg\phi}\ \neg\neg_i \qquad \frac{\phi \to \psi \qquad \neg\psi}{\neg\phi}\ \mathrm{MT} \qquad \frac{\begin{array}{|c|}\hline \neg\phi \\ \vdots \\ \bot \\ \hline \end{array}}{\phi}\ \mathrm{PBC} \qquad \frac{}{\phi \vee \neg\phi}\ \mathrm{LEM}$$

Présentation

- Une preuve est une suite de lignes numérotées. Chaque ligne est une *prémisse* ou résulte d'une règle appliquée à des lignes précédentes.
- Une hypothèse (x_0 frais, hypothèse de $\to_i$, de $\neg_i$, ...) ouvre une boîte. Les lignes de la boîte ne sont plus utilisables une fois qu'elle est fermée.
- Dans ces slides, l'*indentation* marque les boîtes.

Règles pour l'égalité

$$\frac{}{t = t}\ =_i \qquad\qquad \frac{t_1 = t_2 \qquad \phi[t_1/x]}{\phi[t_2/x]}\ =_e$$

Pour appliquer $=_e$, on choisit une formule ϕ dont x marque les positions où l'on remplace t_1 par t_2, avec t_1 et t_2 libres pour x dans ϕ (sinon, de $y = z$ et $\forall y\,(y = y)$, on tirerait $\forall y\,(z = y)$ avec $\phi \equiv \forall y\,(x = y)$).

Exemple 26 (symétrie : $t_1 = t_2 \vdash t_2 = t_1$, avec $\phi \equiv (x = t_1)$)

1.	$t_1 = t_2$	prémisse
2.	$t_1 = t_1$	$=_i$
3.	$t_2 = t_1$	$=_e$ 1, 2

Exemple 27 (congruence : $t_1 = t_2 \vdash f(t_1) = f(t_2)$, avec $\phi \equiv (f(t_1) = x)$)

1.	$t_1 = t_2$	prémisse
2.	$f(t_1) = f(t_1)$	$=_i$
3.	$f(t_1) = f(t_2)$	$=_e$ 1, 2

Règles pour $\forall$

$$\frac{\forall x\, \phi}{\phi[t/x]}\ \forall_e \qquad\qquad \frac{\begin{array}{|c|}\hline x_0 \\ \vdots \\ \phi[x_0/x] \\ \hline\end{array}}{\forall x\, \phi}\ \forall_i$$

- $\forall_e$: t est un terme quelconque, pourvu qu'il soit libre pour x dans ϕ.
- $\forall_i$: x_0 est une variable fraîche, qui n'apparaît nulle part hors de la boîte. On démontre ϕ pour un x_0 *arbitraire*, sur lequel on ne suppose rien.

Pourquoi « libre pour » ?

De $\forall x\, \exists y\, \neg(x = y)$, vraie dès que le domaine a deux éléments, on ne peut pas tirer $\exists y\, \neg(y = y)$ en prenant $t = y$, car cette formule est insatisfaisable. Le y substitué serait capturé par $\exists y$.

Règles pour $\exists$

$$\frac{\phi[t/x]}{\exists x\,\phi}\,\exists_i \qquad\qquad \frac{\exists x\,\phi \qquad \boxed{\begin{array}{ll} x_0 & \phi[x_0/x] \\ & \vdots \\ & \chi \end{array}}}{\chi}\,\exists_e$$

- $\exists_i$: il suffit d'exhiber un témoin t, libre pour x.
- $\exists_e$: on donne un nom x_0 au témoin dont on sait seulement qu'il existe, puis on montre χ sans rien supposer d'autre sur lui. La variable x_0 doit être fraîche et absente de χ.

Stratégie

- La conclusion indique la dernière règle : pour $\forall x\,\phi$, on ouvre une boîte x_0 ; pour $\phi \to \psi$ ou $\neg\phi$, on suppose ϕ.
- Une prémisse $\exists x\,\phi$ appelle $\exists_e$ (à ouvrir tôt), et une prémisse $\forall x\,\phi$ s'instancie avec $\forall_e$.

Exemples : égalité et quantificateurs

Exemple 28 ($a = b,\ \forall x\,(p(x) \to q(x)),\ p(a) \vdash q(b)$)

1.	$a = b$	prémisse
2.	$\forall x\,(p(x) \to q(x))$	prémisse
3.	$p(a)$	prémisse
4.	$p(a) \to q(a)$	$\forall_e$ 2 $(t = a)$
5.	$q(a)$	$\to_e$ 3, 4
6.	$q(b)$	$=_e$ 1, 5 $(\phi \equiv q(x))$

Exemple 29 ($\forall x\,(p(x) \to q(x)),\ \exists x\,p(x) \vdash \exists x\,q(x)$)

1.	$\forall x\,(p(x) \to q(x))$	prémisse
2.	$\exists x\,p(x)$	prémisse
3.	x_0 frais, $p(x_0)$	hypothèse
4.	$p(x_0) \to q(x_0)$	$\forall_e$ 1
5.	$q(x_0)$	$\to_e$ 3, 4
6.	$\exists x\,q(x)$	$\exists_i$ 5
7.	$\exists x\,q(x)$	$\exists_e$ 2, 3–6

Exemple : $\exists$ et $\vee$

Exemple 30 ($\exists x\,(p(x) \vee q(x)) \vdash \exists x\,p(x) \vee \exists x\,q(x)$)

1.	$\exists x\,(p(x) \vee q(x))$	prémisse
2.	$\quad x_0$ frais, $p(x_0) \vee q(x_0)$	hypothèse
3.	$\quad\quad p(x_0)$	hypothèse
4.	$\quad\quad \exists x\,p(x)$	$\exists_i$ 3
5.	$\quad\quad \exists x\,p(x) \vee \exists x\,q(x)$	$\vee_{i_1}$ 4
6.	$\quad\quad q(x_0)$	hypothèse
7.	$\quad\quad \exists x\,q(x)$	$\exists_i$ 6
8.	$\quad\quad \exists x\,p(x) \vee \exists x\,q(x)$	$\vee_{i_2}$ 7
9.	$\quad \exists x\,p(x) \vee \exists x\,q(x)$	$\vee_e$ 2, 3–5, 6–8
10.	$\exists x\,p(x) \vee \exists x\,q(x)$	$\exists_e$ 1, 2–9

La conclusion ne contient pas x_0, donc $\exists_e$ est bien applicable.

Exemple : $\forall$ et $\vee$

Exemple 31 ($\forall x\, p(x) \vee \forall x\, q(x) \vdash \forall x\, (p(x) \vee q(x))$)

1.	$\forall x\, p(x) \vee \forall x\, q(x)$	prémisse
2.	$\quad \forall x\, p(x)$	hypothèse
3.	$\quad\quad x_0$ frais	hypothèse
4.	$\quad\quad p(x_0)$	$\forall_e$ 2
5.	$\quad\quad p(x_0) \vee q(x_0)$	$\vee_{i_1}$ 4
6.	$\quad \forall x\, (p(x) \vee q(x))$	$\forall_i$ 3–5
7.	$\quad \forall x\, q(x)$	hypothèse
8.	$\quad\quad x_1$ frais	hypothèse
9.	$\quad\quad q(x_1)$	$\forall_e$ 7
10.	$\quad\quad p(x_1) \vee q(x_1)$	$\vee_{i_2}$ 9
11.	$\quad \forall x\, (p(x) \vee q(x))$	$\forall_i$ 8–10
12.	$\forall x\, (p(x) \vee q(x))$	$\vee_e$ 1, 2–6, 7–11

La réciproque n'est pas démontrable. Sur $D = \{1, 2\}$ avec $p^{\mathcal{M}} = \{1\}$ et $q^{\mathcal{M}} = \{2\}$, $\forall x\, (p(x) \vee q(x))$ est vraie alors que $\forall x\, p(x) \vee \forall x\, q(x)$ est fausse.

Exemples : négation, $\exists\forall$ vers $\forall\exists$

Exemple 32 ($\exists x\, \neg p(x) \vdash \neg\forall x\, p(x)$)

1.	$\exists x\, \neg p(x)$	prémisse
2.	$\quad \forall x\, p(x)$	hypothèse
3.	$\quad\quad x_0$ frais, $\neg p(x_0)$	hypothèse
4.	$\quad\quad p(x_0)$	$\forall_e$ 2
5.	$\quad\quad \bot$	$\neg_e$ 4, 3
6.	$\quad \bot$	$\exists_e$ 1, 3–5
7.	$\neg\forall x\, p(x)$	$\neg_i$ 2–6

Exemple 33 ($\exists x\forall y\, p(x,y) \vdash \forall y\exists x\, p(x,y)$)

1.	$\exists x\forall y\, p(x,y)$	prémisse
2.	$\quad x_0$ frais, $\forall y\, p(x_0,y)$	hypothèse
3.	$\quad\quad y_0$ frais	hypothèse
4.	$\quad\quad p(x_0,y_0)$	$\forall_e$ 2
5.	$\quad\quad \exists x\, p(x,y_0)$	$\exists_i$ 4
6.	$\quad \forall y\exists x\, p(x,y)$	$\forall_i$ 3–5
7.	$\forall y\exists x\, p(x,y)$	$\exists_e$ 1, 2–6

Une preuve *fausse* : pourquoi x_0 doit être frais

On voudrait « démontrer » $\exists x\, p(x) \vdash \forall x\, p(x)$:

1.	$\exists x\, p(x)$	prémisse
2.	x_0 frais, $p(x_0)$	hypothèse ($\exists_e$)
3.	$\forall x\, p(x)$	$\forall_i$???

- La ligne 3 est interdite : $\forall_i$ exige un x_0 arbitraire, alors qu'ici x_0 est le témoin *particulier* qui vérifie p.
- Le séquent est d'ailleurs faux : prendre $D = \{1, 2\}$ et $p^{\mathcal{M}} = \{1\}$.
- De même, dans la réciproque $\forall y \exists x\, p(x, y) \vdash \exists x \forall y\, p(x, y)$, $\forall_i$ sur y_0 serait bloqué, car y_0 apparaîtrait dans une hypothèse ouverte. Contre-exemple : $\mathbb{N}$ avec p interprété par $\geq$.

Théorème 34 (admis)

La déduction naturelle est correcte, car on ne prouve que des conséquences logiques, et complète, car toute conséquence logique est prouvable.

Plan

1. Syntaxe
2. Sémantique
3. Déduction naturelle
4. Résolution en logique propositionnelle
5. Résolution au premier ordre : formes normales
6. Unification
7. Résolution au premier ordre

L'idée de la résolution

- Une méthode pour tester la satisfaisabilité d'un ensemble de formules.
- Une seule règle, la coupure, appliquée à des clauses.
- Des preuves par réfutation : on dérive la clause vide $\bot$ pour montrer que l'ensemble de départ est insatisfaisable.
- La méthode est adéquate, puisque dériver $\bot$ garantit l'insatisfaisabilité, et complète, puisqu'un ensemble insatisfaisable permet toujours de dériver $\bot$.
- Elle est à la base de la programmation logique (Prolog) et des solveurs SAT.

Littéraux et clauses

Définition 35

- Un littéral est une proposition p ou sa négation $\neg p$.
- Une clause est une disjonction de littéraux $\ell_1 \vee \cdots \vee \ell_n$, ou la clause vide $\bot$.
- On la voit aussi comme un couple (N, P) : N est l'ensemble des propositions qui y apparaissent négativement, P celui des propositions qui y apparaissent positivement.

$\neg a_1 \vee \cdots \vee \neg a_m \vee b_1 \vee \cdots \vee b_n$ se lit $a_1 \wedge \cdots \wedge a_m \rightarrow b_1 \vee \cdots \vee b_n$.

Exemple 36 (catégories)

- $\neg u \vee \neg w$ est une clause *négative*, et $u \vee v \vee w$ une clause *positive*.
- $u \vee \neg v \vee v$ est *tautologique* : elle contient v et $\neg v$.
- $\neg u \vee \neg w \vee v$ se lit $u \wedge w \rightarrow v$.
- $u \wedge v$ n'est *pas* une clause.

Satisfaction d'ensembles de clauses

Définition 37

Un ensemble S de clauses est satisfait par V si $V \models C$ pour toute $C \in S$. Les notions de satisfaisabilité, de validité et de conséquence ($S \models C$) s'étendent de la même façon.

- La clause vide $\bot$ n'est satisfaite par aucune valuation. C'est la seule clause insatisfaisable.
- Une clause tautologique est satisfaite par toutes les valuations. On peut donc retirer les clauses tautologiques sans changer la satisfaisabilité.
- L'ensemble *vide* de clauses est valide.

Proposition 38

Toute formule propositionnelle est équivalente à un ensemble fini de clauses (mise sous FNC).

Exemple 39

$u \leftrightarrow (v \vee w) \equiv (\neg u \vee v \vee w) \wedge ((\neg v \wedge \neg w) \vee u)$, ce qui donne l'ensemble de clauses $\{\neg u \vee v \vee w,\ \neg v \vee u,\ \neg w \vee u\}$.

La règle de coupure

Définition 40 (Coupure sur p)

$$\frac{C_1' \vee p \qquad \neg p \vee C_2'}{C_1' \vee C_2'} \vdash_p^c$$

En notation ensembliste, soient $C_1 = (N_1, P_1)$ et $C_2 = (N_2, P_2)$ avec $p \in P_1$ et $p \in N_2$. Le résolvant est

$$C_3 = \big(N_1 \cup (N_2 \setminus \{p\}),\ (P_1 \setminus \{p\}) \cup P_2\big).$$

On écrit $C_1, C_2 \vdash_p^c C_3$. Si $C_1 = p$ et $C_2 = \neg p$, on obtient $C_3 = \bot$.

Exemple 41

- $\neg e \vee b \vee x,\ \neg x \vee a \vee b \ \vdash_x^c\ \neg e \vee a \vee b$
- $\neg x,\ x \vee y \ \vdash_x^c\ y$ et $\neg x,\ x \ \vdash_x^c\ \bot$
- $u \vee v,\ \neg u \vee \neg v \ \vdash_u^c\ v \vee \neg v$ (clause tautologique, inutile). On n'a pas le droit de couper sur u et v en même temps pour obtenir $\bot$: $\{u \vee v, \neg u \vee \neg v\}$ est d'ailleurs satisfaisable.

Validité de la règle de coupure

Théorème 42

Si $C_1, C_2 \vdash_p^c C_3$, *alors* $C_1 \wedge C_2 \models C_3$.

Démonstration.

Écrivons $C_1 = A \vee p$ et $C_2 = \neg p \vee B$, de sorte que $C_3 = A \vee B$. Soit V une valuation qui satisfait C_1 et C_2.

- Si $V(p) = 0$, alors $V \models A$ puisque $V \models A \vee p$. Donc $V \models C_3$.
- Si $V(p) = 1$, alors $V \models B$ puisque $V \models \neg p \vee B$. Donc $V \models C_3$.

□

Élimination des littéraux redondants

Avec $C_1 = a \vee a \vee \neg b \vee c$ et $C_2 = \neg a \vee d$, la coupure sur a donne $a \vee \neg b \vee c \vee d$. Le littéral a est toujours présent, et l'on ne pourra jamais obtenir $\neg b \vee c \vee d$. On ajoute donc la règle

$$\frac{C \vee \ell \vee \ell}{C \vee \ell} \vdash_r$$

qui est inutile si l'on voit les clauses comme des ensembles.

Preuves par coupure

Définition 43

Une preuve par coupure de C à partir de S est une suite $C_1, \ldots, C_n = C$ telle que, pour chaque i, soit $C_i \in S$, soit $C_k, C_l \vdash_p^c C_i$ avec $k, l < i$, soit $C_k \vdash_r C_i$ avec $k < i$. On écrit $S \vdash^c C$.

Une réfutation de S est une preuve de $\bot$ à partir de S.

Théorème 44 (Adéquation)

Si $S \vdash^c C$, alors $S \models C$. En particulier, si S admet une réfutation, alors S est insatisfaisable.

Démonstration.

On montre par induction sur i que $S \models C_i$.

- Si $C_i \in S$, c'est immédiat.
- Si C_i est obtenue par coupure à partir de C_k et C_l, la validité de la coupure et l'hypothèse d'induction donnent $S \models C_i$.
- Pour $\vdash_r$: $C \vee \ell \vee \ell \equiv C \vee \ell$.

Si $C = \bot$, un modèle de S serait un modèle de $\bot$: impossible. □

Exemples de réfutations

Exemple 45 ($S = \{x,\ \neg x \vee y,\ \neg y \vee z,\ \neg z\}$)

5. y coupure sur x, $1, 2$
6. z coupure sur y, $3, 5$
7. $\bot$ coupure sur z, $4, 6$

Prouver une conséquence par réfutation

Pour montrer $\psi_1, \ldots, \psi_n \models \phi$:

1. on met $\psi_1 \wedge \cdots \wedge \psi_n$ sous forme d'un ensemble de clauses S ;
2. on met $\neg\phi$ sous forme d'un ensemble de clauses S' ;
3. on dérive $\bot$ à partir de $S \cup S'$, et l'on peut s'arrêter dès que $\bot$ apparaît.

Exemple 46 ($u \to v,\ v \to w \models u \to w$)

On part de $\{\neg u \vee v,\ \neg v \vee w,\ u,\ \neg w\}$. La coupure sur u donne v, la coupure sur v donne w, et la coupure sur w donne $\bot$.

Un lemme clé

Exemple 47 (un ensemble satisfaisable)

Pour $S = \{u \vee v,\ \neg u \vee w\}$, la seule coupure possible donne $v \vee w$, et plus rien de nouveau ne peut être produit. La saturation s'arrête sans $\bot$: S est satisfaisable, par exemple avec $u = 0$ et $v = 1$.

Lemme 48

Si S est insatisfaisable et ne contient pas $\bot$, alors une proposition p apparaît positivement dans une clause de S et négativement dans une autre.

Démonstration.

Supposons le contraire : chaque proposition apparaît toujours positivement, ou toujours négativement. Posons $V(x) = 1$ si x apparaît toujours positivement, et $V(x) = 0$ sinon. Chaque clause de S est non vide et contient donc un littéral, qui est vrai sous V. Ainsi $V \models S$, ce qui contredit l'hypothèse. □

Conséquence : la règle de coupure est applicable.

Pas de résolution

Définition 49

Soient S un ensemble de clauses sans clause tautologique et p une proposition.

- $\mathit{PropIn}(S)$ est l'ensemble des propositions qui apparaissent dans S.
- S_p est l'ensemble des clauses de S qui contiennent p.
- $\mathit{Res}(S, p)$ est l'ensemble des résolvants sur p de deux clauses de S_p, et $\mathit{RRes}(S, p)$ l'ensemble de ceux qui ne sont pas tautologiques.
- Le pas de résolution sur p remplace S par

$$S' = (S \setminus S_p) \cup \mathit{RRes}(S, p).$$

Exemple 50

Partons de $S = \{u \vee v,\ \neg u \vee w,\ \neg v,\ \neg w\}$.

- Pas sur u : on obtient $\{v \vee w,\ \neg v,\ \neg w\}$.
- Pas sur v : on obtient $\{w,\ \neg w\}$.
- Pas sur w : on obtient $\{\bot\}$.

Propriétés du pas de résolution

Proposition 51

Si S' s'obtient à partir de S (sans clause tautologique) par un pas de résolution sur p, alors

1. *S' ne contient pas de clause tautologique ;*
2. *S est satisfaisable ssi S' est satisfaisable ;*
3. *$PropIn(S') \subseteq PropIn(S) \setminus \{p\}$.*

Preuve de 1 et 3.

(1) $S \setminus S_p$ n'a pas de clause tautologique, et l'on n'a gardé que les résolvants non tautologiques. (3) Aucune clause de S ne contient à la fois p et $\neg p$, puisqu'elle serait tautologique. Un résolvant de $A \vee p$ et $\neg p \vee B$ est $A \vee B$, qui ne contient donc plus p. Enfin, $S \setminus S_p$ ne contient pas p par définition. □

Propriétés du pas de résolution (suite)

Preuve de 2.

($\Rightarrow$) Si $V \models S$, alors V satisfait les clauses de $S \setminus S_p$ et, par validité de la coupure, tous les résolvants. Donc $V \models S'$.

($\Leftarrow$) Soit $V \models S'$. Écrivons les clauses de S_p sous la forme $p \vee A_i$ et $\neg p \vee B_j$. On définit V sur p, qui n'apparaît pas dans S', comme suit.

- Si $V \models A_i$ pour tout i, on pose $V(p) = 0$. Les clauses $p \vee A_i$ sont vraies grâce à A_i, et les clauses $\neg p \vee B_j$ grâce à $\neg p$.
- Sinon, il existe i_0 tel que $V \not\models A_{i_0}$. Pour tout j, la clause $A_{i_0} \vee B_j$ est soit dans S', soit tautologique : dans les deux cas elle est vraie, donc $V \models B_j$. On pose alors $V(p) = 1$, et toutes les clauses de S_p sont vraies.

Les clauses de $S \setminus S_p$ appartiennent à S' et restent vraies. Donc $V \models S$. □

Complétude de la résolution propositionnelle

Théorème 52

Si S est insatisfaisable, alors il existe une réfutation de S par coupure.

Démonstration.

Par induction sur le nombre n de propositions de S.

- Soit $T \subseteq S$ l'ensemble des clauses non tautologiques. T est insatisfaisable.
- Si $n = 0$, alors $T = \{\bot\}$ et la réfutation est immédiate.
- Si $n > 0$ et $\bot \notin T$, le lemme fournit une proposition p qui apparaît avec les deux signes. Le pas de résolution sur p donne T', insatisfaisable et avec au plus $n - 1$ propositions.
- Par hypothèse d'induction, il existe une réfutation de T'. Comme chaque clause de T' est dans T ou est un résolvant de clauses de T, on en tire une réfutation de T, donc de S.

□

Une procédure de décision

Décision

En logique propositionnelle, saturer par coupure et réduction termine toujours, puisqu'il n'y a qu'un nombre fini de clauses sur un ensemble fini de propositions.

- Si $\bot$ apparaît, S est insatisfaisable (adéquation).
- Sinon, S est satisfaisable (complétude).

La résolution est donc une procédure de décision pour la satisfaisabilité propositionnelle.

Plan

1. Syntaxe
2. Sémantique
3. Déduction naturelle
4. Résolution en logique propositionnelle
5. Résolution au premier ordre : formes normales
6. Unification
7. Résolution au premier ordre

Objectif

Pour établir $\phi_1, \ldots, \phi_n \models \phi$, on montre que $\{\phi_1, \ldots, \phi_n, \neg\phi\}$ est insatisfaisable (proposition 21).

Exemple 53 (l'idée, sans formalisme)

« Tout chat est un mammifère » et « Tom est un chat » entraînent « Tom est un mammifère ». On part de

$$\{\forall x\,(\mathit{chat}(x) \to \mathit{mam}(x)),\ \mathit{chat}(\mathit{tom}),\ \neg\mathit{mam}(\mathit{tom})\}.$$

- La première formule vaut pour tout x, donc en particulier pour $x = \mathit{tom}$.
- Une coupure sur $\mathit{chat}(\mathit{tom})$ donne $\mathit{mam}(\mathit{tom})$, qui contredit la troisième formule.

Au premier ordre, tout le travail consiste à trouver *automatiquement* la bonne instanciation $x \mapsto \mathit{tom}$: c'est le rôle de l'unification.

Méthode pas à pas

1. **Négation.** On part de $\Sigma = \{\phi_1, \ldots, \phi_n, \neg\phi\}$.
2. **Forme prénexe** (formule équivalente). On élimine $\leftrightarrow$ (et éventuellement $\rightarrow$), on renomme les variables liées, puis on fait remonter les quantificateurs.
3. **Skolémisation** (préserve seulement la satisfaisabilité). Chaque $\exists y$ précédé de $\forall x_1 \ldots \forall x_k$ est remplacé par un terme $f(x_1, \ldots, x_k)$, avec f nouveau.
4. **FNC.** On met la matrice sous forme normale conjonctive, puis on distribue $\forall$ sur $\wedge$.
5. **Clauses.** Chaque conjonction donne une clause, et l'on renomme les variables pour que deux clauses n'en partagent aucune.
6. **Résolution** avec unification, jusqu'à obtenir $\bot$.

Forme prénexe

Définition 54

Une formule est prénexe si elle s'écrit $Q_1 x_1 \dots Q_n x_n \cdot \psi$, où les $Q_i \in \{\forall, \exists\}$, les x_i sont distincts et ψ est *sans quantificateur*. Elle est universelle si tous les Q_i sont $\forall$.

Théorème 55

Toute formule est équivalente à une formule prénexe.

Idée de preuve.

Par induction sur la formule, avec les équivalences utiles vues plus haut.

- $\neg$: $\neg Q_1 x_1 \dots Q_n x_n \, \psi \equiv \overline{Q_1} x_1 \dots \overline{Q_n} x_n \, \neg\psi$, en échangeant $\forall$ et $\exists$.
- $\wedge, \vee$: on renomme au besoin la variable quantifiée pour qu'elle ne soit pas libre dans l'autre membre, puis on la fait sortir, et l'on itère.
- $\rightarrow, \leftrightarrow$: on les réécrit avec $\neg, \vee, \wedge$.



Forme prénexe : exemples

Exemple 56

$$\forall x \forall y \left((r(x,y) \wedge \neg x = y) \rightarrow \exists z \left(y = g(x, h(z,z))\right)\right)$$

$$\begin{aligned} &\equiv \forall x \forall y \left((\neg r(x,y) \vee x = y) \vee \exists z \left(y = g(x, h(z,z))\right)\right) \\ &\equiv \forall x \forall y \exists z \left(\neg r(x,y) \vee x = y \vee y = g(x, h(z,z))\right) \end{aligned}$$

Exemple 57 (un $\forall$ dans l'antécédent devient $\exists$)

$$(\forall x\, p(x)) \rightarrow \exists y\, q(y) \;\equiv\; \exists x \left(p(x) \rightarrow \exists y\, q(y)\right) \;\equiv\; \exists x \exists y \left(p(x) \rightarrow q(y)\right)$$

Exemple 58 (l'ordre de sortie est libre)

$\forall x\, p(x) \wedge \exists y\, q(y)$ peut s'écrire $\forall x \exists y \left(p(x) \wedge q(y)\right)$ ou $\exists y \forall x \left(p(x) \wedge q(y)\right)$. Les deux formes sont équivalentes, mais la seconde donnera une constante de Skolem au lieu d'une fonction. Il est donc souvent plus simple de sortir d'abord les $\exists$.

Skolémisation

Définition 59

- Une extension de Skolem de $\mathcal{L}$ lui ajoute une infinité de nouveaux symboles de fonction de chaque arité et une infinité de nouvelles constantes.
- Un pas de skolémisation transforme la formule close prénexe

$$\forall x_1 \ldots \forall x_k \, \exists x \, \psi \quad \text{en} \quad \forall x_1 \ldots \forall x_k \, \psi[f(x_1, \ldots, x_k)/x],$$

  avec f nouveau. Si $k = 0$, on remplace x par une constante nouvelle c.
- En itérant ce pas jusqu'à ce qu'il ne reste plus aucun $\exists$, on obtient une forme de Skolem. Les symboles introduits sont les *fonctions et constantes de Skolem*.

Exemple 60

$\exists x \, \forall y \, \exists z \, \forall u \, \exists w \; r(x, y, z, u, w)$ devient $\forall y \, \forall u \; r(c, y, f(y), u, g(y, u))$. Le terme z dépend de y seulement, et w dépend de y et u, mais jamais de x, qui est existentiel.

Pourquoi une « approximation » ?

$\psi \equiv \forall x \, \exists y \, p(x, y)$ dit : pour chaque x, on peut *choisir* un y. Sa forme de Skolem $\psi' \equiv \forall x \, p(x, f(x))$ fixe ce choix, $y = f(x)$.

Exemple 61

Prenons $M = \{1, 2\}$ et interprétons f par $1 \mapsto 2$ et $2 \mapsto 1$.

- Si $p^{\mathcal{M}_1} = \{(1,1), (1,2), (2,1), (2,2)\}$, alors $\mathcal{M}'_1 \models \psi'$.
- Si $p^{\mathcal{M}_2} = \{(1,1), (2,2)\}$, alors $\mathcal{M}_2 \models \psi$ mais $\mathcal{M}'_2 \not\models \psi'$. Il aurait fallu interpréter f par l'identité.

- On remplace une dépendance *relationnelle* par une dépendance *fonctionnelle*, et l'on doit pouvoir choisir cette fonction.
- Cas le plus simple : $\exists x \, p(x)$ et $p(c)$ ne sont pas équivalentes. La structure $D = \{1, 2\}$, $p^{\mathcal{M}} = \{1\}$, $c^{\mathcal{M}} = 2$ satisfait la première mais pas la seconde.

La skolémisation préserve la satisfaisabilité

Théorème 62

Si ϕ_1 est obtenue à partir de ϕ par un pas de skolémisation, alors ϕ est satisfaisable ssi ϕ_1 est satisfaisable.

Démonstration.

Soient $\phi \equiv \forall x_1 \ldots \forall x_k \, \exists x \, \psi$ et $\phi_1 \equiv \forall x_1 \ldots \forall x_k \, \psi[f(x_1, \ldots, x_k)/x]$.

($\Leftarrow$) La formule $\forall \vec{x} \, \psi[f(\vec{x})/x] \rightarrow \forall \vec{x} \, \exists x \, \psi$ est valide : pour chaque $\vec{a}$, l'élément $f^{\mathcal{M}}(\vec{a})$ est un témoin. Tout modèle de ϕ_1 est donc un modèle de ϕ.

($\Rightarrow$) Soit $\mathcal{M} \models \phi$. Pour tout $\vec{a} = (a_1, \ldots, a_k) \in M^k$, il existe $b \in M$ tel que $\mathcal{M}, [\vec{x} \mapsto \vec{a}, x \mapsto b] \models \psi$. On construit $\mathcal{M}'$ en posant $f^{\mathcal{M}'}(\vec{a}) = b$, un tel b étant choisi pour chaque $\vec{a}$. Alors $\mathcal{M}' \models \phi_1$. Si $k = 0$, on pose $c^{\mathcal{M}'} = b$. □

En revanche, l'équivalence n'est pas préservée, ni la validité.

Forme clausale

Définition 63

- Un littéral est une formule atomique ou sa négation.
- Une clause est une disjonction de littéraux.
- Une formule prénexe $Q_1 x_1 \dots Q_n x_n \, \psi$ est en forme normale conjonctive (respectivement disjonctive) si ψ est une conjonction de clauses (respectivement une disjonction de conjonctions de littéraux).

Théorème 64

Toute formule est équivalente à une formule prénexe en FNC.

Transformation d'un ensemble Σ de formules closes

Forme prénexe, puis skolémisation $\forall x_1 \dots \forall x_n \, \phi_s$, puis FNC $\phi_s \equiv \bigwedge_i C_i$. Comme $\forall x \, (\psi \wedge \psi') \equiv \forall x \, \psi \wedge \forall x \, \psi'$, chaque C_i donne une clause, dont les variables sont implicitement quantifiées universellement.

Théorème 65

L'ensemble de clauses S obtenu est satisfaisable ssi Σ est satisfaisable.

Forme clausale : un exemple complet

Exemple 66

On veut montrer $\forall x \left(e(x) \to \exists y \left(c(y) \wedge s(x,y)\right)\right),\ e(a) \models \exists y\, c(y)$, où e signifie « est étudiant », c « est un cours » et s « suit ».

1. On ajoute la négation de la conclusion, $\neg\exists y\, c(y)$, aux prémisses.
2. Forme prénexe : $\forall x\, \exists y \left(\neg e(x) \vee \left(c(y) \wedge s(x,y)\right)\right)$ et $\forall y\, \neg c(y)$.
3. Skolémisation, avec $y \mapsto f(x)$: $\forall x \left(\neg e(x) \vee \left(c(f(x)) \wedge s(x, f(x))\right)\right)$.
4. FNC : $\left(\neg e(x) \vee c(f(x))\right) \wedge \left(\neg e(x) \vee s(x, f(x))\right)$.
5. Clauses, après renommage : $C_1 = \neg e(x_1) \vee c(f(x_1))$, $C_2 = \neg e(x_2) \vee s(x_2, f(x_2))$, $C_3 = e(a)$, $C_4 = \neg c(y)$.
6. Réfutation : C_1 et C_3 avec $x_1 \mapsto a$ donnent $c(f(a))$, puis $c(f(a))$ et C_4 avec $y \mapsto f(a)$ donnent $\bot$.

La clause C_2 n'a pas servi : une réfutation n'utilise pas forcément toutes les clauses.

Mise en forme clausale : l'algorithme

FormeClausale(Σ)

Entrée : un ensemble fini Σ de formules closes ($\neg\phi$ déjà ajoutée).
Sortie : un ensemble de clauses S, satisfaisable ssi Σ l'est.

1. $S \leftarrow \emptyset$; **pour chaque** $\phi \in \Sigma$ **faire**
   1. *Connecteurs :* $\alpha \leftrightarrow \beta \rightsquigarrow (\alpha \to \beta) \wedge (\beta \to \alpha)$, puis $\alpha \to \beta \rightsquigarrow \neg\alpha \vee \beta$. [$\equiv$]
   2. *Renommage :* variables liées deux à deux distinctes, et distinctes des variables libres. [$\equiv$]
   3. *Prénexe :* faire remonter les quantificateurs ($\neg\forall x \rightsquigarrow \exists x\,\neg$, $\neg\exists x \rightsquigarrow \forall x\,\neg$, $(Qx\,\alpha) \circ \beta \rightsquigarrow Qx\,(\alpha \circ \beta)$ pour $\circ \in \{\wedge, \vee\}$). On obtient $Q_1 x_1 \ldots Q_n x_n\,\psi$, avec ψ sans quantificateur. [$\equiv$]
   4. *Skolémisation :* **pour** $i = 1, \ldots, n$, **si** $Q_i = \exists$: soient $x_{j_1}, \ldots, x_{j_k}$ les variables des $\forall$ *qui précèdent* x_i ; remplacer x_i par $f(x_{j_1}, \ldots, x_{j_k})$, avec f *nouveau* (une constante si $k = 0$), et supprimer $\exists x_i$. [sat.]
   5. *FNC :* mettre ψ sous la forme $C_1 \wedge \cdots \wedge C_m$ (De Morgan, $\neg\neg$, distributivité). [$\equiv$]
   6. *Clauses :* $S \leftarrow S \cup \{C_1, \ldots, C_m\}$ (car $\forall\vec{x}\,(C_1 \wedge \cdots \wedge C_m) \equiv \bigwedge_j \forall\vec{x}\,C_j$).
2. renommer les variables pour que deux clauses de S n'en partagent aucune ; **retourner** S.

[$\equiv$] : préserve l'équivalence ; [sat.] : seulement la satisfaisabilité. Symboles de Skolem nouveaux *pour tout* Σ, pas seulement pour ϕ.

Plan

1. Syntaxe
2. Sémantique
3. Déduction naturelle
4. Résolution en logique propositionnelle
5. Résolution au premier ordre : formes normales
6. Unification
7. Résolution au premier ordre

Substitutions

Définition 67

- Une substitution α est une fonction des variables dans les termes, de domaine $\{x \mid \alpha(x) \neq x\}$ fini. On la note $(t_1/x_1, \ldots, t_n/x_n)$.
- Elle s'étend aux termes de manière unique : $\alpha(c) = c$ et $\alpha(f(t_1, \ldots, t_n)) = f(\alpha(t_1), \ldots, \alpha(t_n))$.
- Elle s'étend aussi aux formules sans quantificateur, donc aux clauses, en remplaçant *simultanément* chaque x_i par t_i.

Si t est clos, alors $\alpha(t) = t$.

Exemple 68

Pour $\alpha = (g(d, z)/x,\ f(z)/y,\ d/z)$ et $C \equiv r(x, f(y)) \vee \neg r(g(y, c), z)$:

$$\alpha(C) = r(g(d, z), f(f(z))) \vee \neg r(g(f(z), c), d).$$

La substitution est simultanée : le z introduit pour x n'est pas remplacé à son tour par d.

Unificateurs

Définition 69

- α est un unificateur de t et t' si $\alpha(t) = \alpha(t')$, et d'un système (ensemble fini de couples) $S = \{(t_i, t'_i)\}_i$ s'il unifie chacun de ses couples.
- Un ensemble d'atomes $\{A_1, \ldots, A_n\}$ est unifiable s'il existe α tel que $\alpha(A_1) = \cdots = \alpha(A_n)$.
- σ est un unificateur principal (mgu) si, pour tout unificateur α, il existe β tel que $\alpha = \beta \circ \sigma$.

Exemple 70

$p(x, y)$ et $p(y, x)$ sont unifiés par $\sigma = (y/x)$, qui donne $p(y, y)$, et par $\alpha = (a/x, a/y)$, qui donne $p(a, a)$.

- $\alpha = (a/y) \circ \sigma$: σ est plus général, et c'est un mgu.
- α n'est pas principal, car on ne peut pas retrouver σ à partir de lui.

Hauteur d'un terme

Définition 71

On pose $\mathcal{T}_0$ l'ensemble des constantes et des variables. Un terme t appartient à $\mathcal{T}_{n+1}$ si $t \in \mathcal{T}_n$ ou si $t = f(t_1, \ldots, t_k)$ avec tous les $t_i \in \mathcal{T}_n$. La hauteur de t est le plus petit n tel que $t \in \mathcal{T}_n$. Autrement dit, elle vaut 0 pour une constante ou une variable, et $1 + \max_i \mathrm{h}(t_i)$ pour $f(t_1, \ldots, t_k)$.

Proposition 72

Pour toute substitution α et tout terme t, $\mathrm{h}(\alpha(t)) \geq \mathrm{h}(t)$.

Démonstration.

Par induction sur t. Pour une constante ou une variable, $\mathrm{h}(t) = 0$. Pour $t = f(t_1, \ldots, t_k)$, on a $\mathrm{h}(\alpha(t)) = 1 + \max_i \mathrm{h}(\alpha(t_i)) \geq 1 + \max_i \mathrm{h}(t_i) = \mathrm{h}(t)$. □

Corollaire 73

Si x apparaît dans t et $t \neq x$, alors x et t ne sont pas unifiables. En effet, $\sigma(x)$ serait un sous-terme strict de $\sigma(t)$, donc de hauteur strictement plus petite.

Unicité de l'unificateur principal

Théorème 74

Si σ et σ' sont deux unificateurs principaux de t et t', alors ils sont égaux à un renommage (permutation) des variables près.

Démonstration.

Comme les deux sont principaux, il existe γ et β tels que $\sigma = \gamma \circ \sigma'$ et $\sigma' = \beta \circ \sigma$. Donc $\sigma = \gamma \circ \beta \circ \sigma$: $\gamma \circ \beta$ fixe tous les termes $\sigma(x)$.

- β envoie chaque variable y de ces termes sur une *variable*. Sinon, $\mathrm{h}(\beta(y)) \geq 1$, et γ ne pourrait pas faire redescendre la hauteur à 0 pour retrouver y. De même, β ne peut pas envoyer y sur une constante, car ce serait irréversible.
- β est injective sur ces variables, puisque $\gamma \circ \beta$ est l'identité sur elles.

Donc β est un renommage de variables, et de même pour γ. □

Systèmes et simplification

Un système est un ensemble fini de couples de termes. On cherche ses unificateurs.

Lemme 75 (Simplification)

Soit un couple $(f(t_1, \ldots, t_n),\ g(t'_1, \ldots, t'_m))$.

- *Si* $f \neq g$, *il n'est pas unifiable, et le système non plus.*
- *Si* $f = g$ *(donc* $n = m$*), on peut le remplacer par* $(t_1, t'_1), \ldots, (t_n, t'_n)$, *qui ont les mêmes unificateurs.*

On peut aussi supprimer les couples (t, t) *et les doublons.*

En itérant, on obtient soit un échec, soit un système simplifié, où chaque couple contient au moins une variable ou une constante.

Lemme 76 (Couples irréductibles)

Un couple d'un système simplifié est réductible s'il est de la forme (x, t) *ou* (t, x) *avec* x *absent de* t. *Sinon, il est irréductible, et le système n'est pas unifiable :*

- (c, d) *avec* $c \neq d$ *deux constantes, ou* $(f(\ldots), c)$ *: les symboles de tête diffèrent ;*
- $(x, f(\ldots x \ldots))$ *: impossible pour une raison de hauteur.*

Algorithme d'unification

Définition 77

Un système est en forme résolue s'il est de la forme $\{(x_1, t_1), \ldots, (x_n, t_n)\}$, où les x_i sont des variables distinctes qui n'apparaissent nulle part ailleurs dans le système. Il définit la substitution $(t_1/x_1, \ldots, t_n/x_n)$.

Algorithme : appliquer la première règle possible, et recommencer

1. **Orienter** : (t, x), où t n'est pas une variable, devient (x, t).
2. **Effacer** : on supprime les couples (x, x).
3. **Simplifier** : on décompose $(f(\vec{t}), f(\vec{t'}))$, avec échec si les symboles de tête diffèrent.
4. **Éliminer** : si (x, t) appartient au système et que x apparaît ailleurs, on conclut à un échec si x apparaît dans t (test d'occurrence). Sinon, on remplace x par t dans tous les *autres* couples.

Si aucune règle ne s'applique et qu'il n'y a pas eu d'échec, on retourne la substitution définie par la forme résolue.

Exemples

Exemple 78 (succès : $p(x, f(y), a)$ et $p(g(z), f(g(z)), z)$)

$$\{(x, g(z)),\ (f(y), f(g(z))),\ (a, z)\} \quad \text{simplification de } p$$
$$\{(x, g(z)),\ (y, g(z)),\ (z, a)\} \quad \text{simplification de } f, \text{ orientation}$$
$$\{(x, g(a)),\ (y, g(a)),\ (z, a)\} \quad \text{élimination de } z$$

Le mgu est $(g(a)/x,\ g(a)/y,\ a/z)$.

Exemple 79 (échec subtil : $f(x, x)$ et $f(y, g(y))$)

La simplification donne $\{(x, y), (x, g(y))\}$. L'élimination de x donne $\{(x, y), (y, g(y))\}$, et l'on obtient un échec par le test d'occurrence. Chaque couple de départ était unifiable seul : c'est leur combinaison qui ne l'est pas.

Exemple 80 (autres échecs)

x et $f(x)$ (test d'occurrence), ou $g(x, c)$ et $g(d, x)$, qui imposeraient $c = d$. En revanche, $g(x, c)$ et $g(d, y)$ s'unifient par $(d/x,\ c/y)$.

Un exemple plus long

Exemple 81

Partons de $S_1 = \{(f(c, g(x_4, x_5)), x_2),\ (f(c, g(x_5, x_4)), x_2),\ (x_2, x_6)\}$.

- On oriente : $\{(x_2, f(c, g(x_4, x_5))),\ (x_2, f(c, g(x_5, x_4))),\ (x_6, x_2)\}$.
- On élimine x_2 avec le premier couple :
  $\{(x_2, f(c, g(x_4, x_5))),\ (f(c, g(x_4, x_5)), f(c, g(x_5, x_4))),\ (x_6, f(c, g(x_4, x_5)))\}$.
- On simplifie : le deuxième couple donne (x_4, x_5) et (x_5, x_4).
- On élimine x_4 : (x_5, x_4) devient (x_5, x_5), que l'on efface.

On obtient la forme résolue

$$\{(x_2, f(c, g(x_5, x_5))),\ (x_4, x_5),\ (x_6, f(c, g(x_5, x_5)))\}.$$

Terminaison

Théorème 82

L'algorithme termine. S'il échoue, le système n'est pas unifiable. S'il réussit, le système obtenu est en forme résolue et définit un unificateur principal du système de départ.

Preuve de la terminaison.

- Sans la règle 4, les règles 1 à 3 ne peuvent s'appliquer qu'un nombre fini de fois d'affilée : chacune diminue la taille du système ou le nombre de couples mal orientés.
- Soit m le nombre de variables du système. Après une application de la règle 4 sur (x, t), x n'apparaît plus que dans ce couple, et cela reste vrai par la suite, car les autres règles n'introduisent pas x ailleurs. La règle 4 ne s'applique donc qu'au plus une fois par variable, soit au plus m fois.

□

Correction

Chaque règle préserve l'ensemble des unificateurs.

Pour les règles 1 et 2, c'est clair. Pour la règle 3, c'est le lemme de simplification.

Pour la règle 4, soit $\beta = (t/x)$, et supposons que $(u_1, u_2) = (\beta(t_1), \beta(t_2))$ pour un couple $(t_1, t_2) \neq (x, t)$ du système.

- Tout unificateur σ vérifie $\sigma(x) = \sigma(t)$, donc $\sigma \circ \beta = \sigma$. En effet, $\sigma(\beta(x)) = \sigma(t) = \sigma(x)$ et $\sigma(\beta(y)) = \sigma(y)$ pour $y \neq x$.
- Par conséquent, $\sigma(u_i) = \sigma(\beta(t_i)) = \sigma(t_i)$, et σ unifie (u_1, u_2) ssi il unifie (t_1, t_2).

Échec. Il ne se produit qu'avec la règle 3 (symboles différents) ou avec la règle 4 (test d'occurrence). Dans les deux cas, le système courant n'a pas d'unificateur, donc le système de départ non plus.

Forme résolue. Si plus aucune règle ne s'applique, chaque couple est de la forme (x, t) avec $t \neq x$ (sinon les règles 1 à 3 s'appliqueraient), et x n'apparaît nulle part ailleurs (sinon la règle 4 s'appliquerait). □

Principalité

La forme résolue définit un unificateur principal.

Soit $S_r = \{(x_1, t_1), \ldots, (x_n, t_n)\}$ la forme résolue et $\sigma = (t_1/x_1, \ldots, t_n/x_n)$.

- σ unifie S_r : $\sigma(x_i) = t_i = \sigma(t_i)$, car aucun x_j n'apparaît dans t_i.
- Soit σ' un unificateur de S_r. Alors $\sigma'(x_i) = \sigma'(t_i) = \sigma'(\sigma(x_i))$ pour tout i.
- Pour $y \notin \{x_1, \ldots, x_n\}$, on a $\sigma(y) = y$, donc $\sigma'(y) = \sigma'(\sigma(y))$.
- Donc $\sigma' = \sigma' \circ \sigma$: σ est principal pour S_r.

Comme S_r et le système de départ ont les mêmes unificateurs, σ est principal pour le système de départ. □

Plan

1. Syntaxe
2. Sémantique
3. Déduction naturelle
4. Résolution en logique propositionnelle
5. Résolution au premier ordre : formes normales
6. Unification
7. Résolution au premier ordre

Avant de commencer : et l'égalité ?

Le prédicat $=$ est toujours interprété par l'identité du domaine, et la déduction naturelle lui donnait ses propres règles ($=_i$, $=_e$). La résolution, elle, ne connaît que l'unification *syntaxique* : elle traite $=$ comme un prédicat binaire quelconque.

Exemple 83 (ce que la résolution seule ne voit pas)

$S = \{a = b,\ p(a),\ \neg p(b)\}$ est insatisfaisable, mais $p(a)$ et $p(b)$ ne s'unifient pas : aucune résolution n'est possible, et l'on n'obtient jamais $\bot$.

Remarque 84

Pour simplifier l'exposé, nous considérons dans la suite des clauses sans égalité. On peut la prendre en compte de deux façons.

- **Ajouter des axiomes** à S, qui disent que $=$ est une congruence : réflexivité, symétrie, transitivité, et compatibilité avec chaque symbole de fonction et de prédicat du langage, par exemple $\neg x = y \vee \neg p(x) \vee p(y)$. L'équivalence seule ne suffit pas. La résolution habituelle devient alors complète, mais très inefficace.
- **Ajouter une règle d'inférence** : la paramodulation (Robinson et Wos, 1969) remplace « égal par égal » en un seul pas, avec unification. Ses raffinements, comme la superposition, sont au cœur des prouveurs automatiques actuels.

La règle de résolution

Soit $C = (N, P)$ une clause : N est l'ensemble de ses atomes *négatifs*, P celui de ses atomes *positifs*.

Définition 85 (Résolvant)

Soient $C_1 = (N_1, P_1)$ et $C_2 = (N_2, P_2)$ deux clauses séparées, c'est-à-dire sans variable commune (on renomme si nécessaire). S'il existe $\varnothing \neq P_1' \subseteq P_1$ et $\varnothing \neq N_2' \subseteq N_2$ tels que $P_1' \cup N_2'$ soit unifiable, de mgu σ, alors le résolvant $C = (N, P)$ est défini par

$$N = \sigma(N_1) \cup \sigma(N_2 \setminus N_2'), \qquad P = \sigma(P_1 \setminus P_1') \cup \sigma(P_2).$$

En deux temps : on *unifie* des atomes positifs de C_1 avec des atomes négatifs de C_2, puis on *coupe* sur l'atome obtenu.

Exemple 86

Pour $C_1 \equiv \neg s(z) \vee p(z) \vee q(z)$ et $C_2 \equiv \neg p(f(y)) \vee r(x, y)$, on unifie $\{p(z), p(f(y))\}$ par $\sigma = (f(y)/z)$. Le résolvant est

$$\neg s(f(y)) \vee q(f(y)) \vee r(x, y).$$

Pourquoi séparer ? Pourquoi des ensembles ?

Exemple 87 (séparer les clauses)

$\{p(x),\ \neg p(f(x))\}$ est insatisfaisable. Sans renommage, unifier $p(x)$ et $p(f(x))$ échoue au test d'occurrence. Après renommage en $p(y)$, la substitution $y \mapsto f(x)$ donne $\bot$. Les deux x étaient des variables indépendantes, chacune quantifiée universellement dans sa propre clause.

Exemple 88 (unifier des ensembles de littéraux)

Prenons $C_1 = p(a,x) \vee p(x,a)$ et $C_2 = \neg p(a,y) \vee \neg p(y,a)$.

- En unifiant un seul littéral de chaque côté, on n'obtient que des clauses du type $p(a,a) \vee \neg p(a,a)$, qui sont tautologiques.
- En unifiant les quatre atomes avec $(a/x, a/y)$, on obtient $\bot$ en une étape.

Variante équivalente : n'unifier que des singletons, mais ajouter la factorisation, qui passe de $\ell_1 \vee \ell_2 \vee C$ à $\sigma(\ell_1) \vee \sigma(C)$ lorsque σ est un mgu de ℓ_1 et ℓ_2.

Preuves par résolution

Définition 89

Une preuve par résolution de C à partir de S est une suite $C_1, \ldots, C_n = C$ telle que chaque C_i appartient à S ou est un résolvant de C_j et C_k avec $j, k < i$. On écrit $S \vdash_R C$. Une réfutation de S est une preuve de la clause vide $(\varnothing, \varnothing)$.

Exemple 90 (de bout en bout)

Montrons $\forall x\,(p(x) \to q(x)),\ \exists x\, p(x) \models \exists x\, q(x)$. Après négation, skolémisation et renommage, on obtient $S = \{\neg p(x_1) \vee q(x_1),\ p(a),\ \neg q(x_2)\}$.

4. $q(a)$ résolution $1, 2$ avec (a/x_1)
5. $\bot$ résolution $3, 4$ avec (a/x_2)

Réfutations : un deuxième exemple

Exemple 91 (une clause utilisée deux fois)

Soit $S = \{\neg p(x) \vee p(f(x)),\ p(a),\ \neg p(f(f(a)))\}$.

4.	$p(f(a))$	résolution $1, 2$ avec (a/x)
5.	$p(f(f(a)))$	résolution $1', 4$ avec $(f(a)/x')$
6.	$\bot$	résolution $3, 5$

Ici, $1' = \neg p(x') \vee p(f(x'))$ est une copie renommée de la clause 1.

Structures de Herbrand

La résolution est purement syntaxique. Pour la justifier, on montre qu'il suffit de chercher des modèles dont le domaine est formé des *termes clos* eux-mêmes.

Définition 92

- Le domaine de Herbrand $\mathcal{H}_D$ est l'ensemble des termes clos de $\mathcal{L}$. Si $\mathcal{L}$ ne contient aucune constante, on en ajoute une.
- La base de Herbrand $\mathcal{H}_B$ est l'ensemble des atomes clos.
- Une clause est close si elle ne contient aucune variable. Une instance close de C est une clause close $\sigma(C)$, où σ envoie chaque variable de C sur un terme clos.
- Une structure de Herbrand a pour domaine $\mathcal{H}_D$ et interprète chaque terme clos par lui-même : $f^{\mathcal{H}}(t_1, \ldots, t_m) = f(t_1, \ldots, t_m)$. La donner revient à choisir $V : \mathcal{H}_B \to \{0, 1\}$. Une telle fonction définie sur une partie seulement de $\mathcal{H}_B$ est une *structure partielle*.
- Un modèle de Herbrand de ϕ est une structure de Herbrand qui satisfait ϕ.

Attention

Clause close : *aucune* variable ; formule close : aucune variable *libre*. $\neg p(x) \vee q(f(x))$ n'est pas close ; $\neg p(a) \vee q(f(a))$ en est une instance close.

Domaines de Herbrand : exemples

Exemple 93

- Pour $\mathcal{L} = \{a, b, p|_1\}$, sans fonction : $\mathcal{H}_D = \{a, b\}$ et $\mathcal{H}_B = \{p(a), p(b)\}$. Tout est fini, et l'on se ramène au cas propositionnel.
- Pour $\mathcal{L} = \{a, g|_2, r|_1\}$: $\mathcal{H}_D = \{a,\ g(a,a),\ g(a,g(a,a)),\ g(g(a,a),a), \dots\}$, qui est infini.
- Pour $\mathcal{L} = \{f|_1, p|_1\}$, sans constante : on ajoute c, et $\mathcal{H}_D = \{c, f(c), f(f(c)), \dots\}$, $\mathcal{H}_B = \{p(f^n(c)) \mid n \geq 0\}$.

Dès qu'il y a un symbole de fonction (et un prédicat), $\mathcal{H}_D$ et $\mathcal{H}_B$ sont infinis dénombrables, le langage étant supposé au plus dénombrable.

Exemple 94 (un modèle de Herbrand)

$\{p(a),\ \forall x\,(p(x) \to p(f(f(x))))\}$ a pour modèle de Herbrand la structure où $p(f^n(a))$ est vrai ssi n est pair. C'est la trace de $(\mathbb{N}, \text{pair}, 0, \mathit{succ})$.

Modèle de Herbrand d'une formule universelle

Lemme 95

Une formule universelle close $\phi \equiv \forall x_1 \ldots \forall x_k \, \psi$ a un modèle ssi elle a un modèle de Herbrand.

Démonstration.

($\Leftarrow$) Un modèle de Herbrand est un modèle.

($\Rightarrow$) Soit $\mathcal{M} \models \phi$. On définit $\mathcal{H}$ en déclarant l'atome clos $p(t_1, \ldots, t_n)$ vrai ssi $\mathcal{M} \models p(t_1, \ldots, t_n)$.

- Par induction sur les formules sans quantificateur, $\mathcal{H} \models \psi(\vec{t})$ ssi $\mathcal{M} \models \psi(\vec{t})$, pour tous termes clos $\vec{t}$.
- Comme $\mathcal{M} \models \forall \vec{x} \, \psi$, on a $\mathcal{M} \models \psi(\vec{t})$ pour tous $\vec{t} \in \mathcal{H}_D$, donc $\mathcal{H} \models \psi(\vec{t})$ pour tous ces $\vec{t}$.
- Le domaine de $\mathcal{H}$ étant $\mathcal{H}_D$, cela signifie exactement $\mathcal{H} \models \forall \vec{x} \, \psi$.

□

Conséquence : un ensemble de clauses sans modèle de Herbrand n'a aucun modèle.

Le théorème de Herbrand

Théorème 96 (Herbrand, admis)

Si un ensemble S de clauses est insatisfaisable, il existe un ensemble fini d'instances closes de clauses de S qui est insatisfaisable, vu comme un ensemble de clauses propositionnelles dont les atomes clos jouent le rôle de propositions.

Exemple 97

Pour $S = \{\neg p(x) \vee p(f(x)),\ p(a),\ \neg p(f(f(a)))\}$, on prend les instances

$$\neg p(a) \vee p(f(a))\ (x \mapsto a), \qquad \neg p(f(a)) \vee p(f(f(a)))\ (x \mapsto f(a)).$$

Avec $p(a)$ et $\neg p(f(f(a)))$, elles forment un ensemble propositionnel insatisfaisable. C'est exactement ce que la réfutation vue plus haut trouve *automatiquement*, grâce à l'unification.

Arbres sémantiques

- On énumère $\mathcal{H}_B = \{A_1, A_2, \dots\}$. L'arbre sémantique est l'arbre binaire dont chaque nœud de niveau $n-1$ a deux fils, étiquetés A_n et $\neg A_n$. Chaque branche est une structure de Herbrand.
- Un nœud est un nœud d'échec si la structure partielle qu'il définit rend fausse une instance close d'une clause de S, et si aucun de ses ancêtres ne le fait déjà.
- L'arbre est fermé si chaque branche contient un nœud d'échec.

Théorème 98

S est insatisfaisable ssi S possède un arbre sémantique fermé.

Exemple 99 ($S = \{p(x),\ \neg p(f(a))\}$, énumération $p(a), p(f(a)), \dots$)

- La branche $\neg p(a)$ échoue au niveau 1, sur l'instance $p(a)$ de $p(x)$.
- Sous $p(a)$, la branche $p(f(a))$ échoue sur $\neg p(f(a))$, et la branche $\neg p(f(a))$ sur l'instance $p(f(a))$ de $p(x)$.

L'arbre est fermé, donc S est insatisfaisable.

Correction : une propriété des substitutions

Lemme 100 (admis, par induction)

Soient $B(x_1, \ldots, x_k)$ *une clause et* σ *une substitution avec* $\sigma(x_i) = t_i(y_1, \ldots, y_l)$. *Pour* $b_1, \ldots, b_l \in M$, *on pose* $a_i = t_i^{\mathcal{M}}[b_1, \ldots, b_l]$. *Alors*

$$\mathcal{M}, [a_1, \ldots, a_k] \models B \quad \textit{ssi} \quad \mathcal{M}, [b_1, \ldots, b_l] \models \sigma(B).$$

Autrement dit, évaluer $\sigma(B)$ avec les b_j revient à évaluer B avec les valeurs des termes substitués.

Notons $\bar{C}$ la clôture universelle de la clause C : on quantifie universellement toutes ses variables.

Adéquation de la règle de résolution

Lemme 101

Si C est un résolvant de C_1 et C_2, alors $\bar{C}_1 \wedge \bar{C}_2 \models \bar{C}$.

Démonstration.

Par contraposée. Supposons que $\mathcal{M} \not\models \bar{C}$: il existe $\vec{b}$ tel que $\mathcal{M}, \vec{b} \models \neg C$. On pose $a_i = \sigma(x_i)^{\mathcal{M}}[\vec{b}]$ pour les variables x_i de C_1 et C_2, qui sont disjointes. Puisque C est fausse pour $\vec{b}$:

- les atomes de $\sigma(N_1)$ et de $\sigma(N_2 \setminus N_2')$ sont *vrais* ;
- les atomes de $\sigma(P_1 \setminus P_1')$ et de $\sigma(P_2)$ sont *faux*.

Soit A l'atome $\sigma(P_1') = \sigma(N_2')$. D'après le lemme précédent, on transporte ces valeurs à C_1 et C_2 évaluées en $\vec{a}$.

- Si A est vrai en $\vec{b}$, les atomes de N_2' sont vrais en $\vec{a}$, ceux de $N_2 \setminus N_2'$ aussi, et ceux de P_2 sont faux. Donc C_2 est fausse en $\vec{a}$ et $\mathcal{M} \not\models \bar{C}_2$.
- Si A est faux en $\vec{b}$, les atomes de P_1' et de $P_1 \setminus P_1'$ sont faux et ceux de N_1 sont vrais en $\vec{a}$. Donc C_1 est fausse en $\vec{a}$ et $\mathcal{M} \not\models \bar{C}_1$.

□

Correction de la résolution

Corollaire 102 (Correction)

Si $S \vdash_R C$, alors $\bar{C}$ est conséquence logique de $\bar{S}$. En particulier, s'il existe une réfutation de S, alors S n'a pas de modèle.

Démonstration.

On raisonne par induction sur la longueur de la preuve, avec le lemme précédent. Si $C = \bot$, un modèle de $\bar{S}$ serait un modèle de $\bot$, ce qui est impossible. □

Complétude : schéma de la preuve

Théorème 103

Si un ensemble de clauses S est insatisfaisable, alors il existe une réfutation de S par résolution.

1. D'après le théorème de Herbrand, il existe un ensemble fini $S_c \subseteq \{\sigma(C) \mid C \in S,\ \sigma \text{ substitution close}\}$ qui est insatisfaisable.
2. D'après la complétude de la résolution propositionnelle, S_c admet une réfutation close.
3. On relève cette réfutation close en une réfutation de S, grâce au lemme du relèvement.

Lemme du relèvement

Lemme 104

Soient C_1', C_2' des instances closes de C_1, C_2, et C' un résolvant (clos) de C_1' et C_2'. Il existe alors un résolvant C de C_1, C_2 tel que C' soit une instance de C.

Démonstration.

On a $C_1' = \sigma_1(C_1)$ et $C_2' = \sigma_2(C_2)$. Quitte à renommer, C_1 et C_2 sont séparées, et $\tau = \sigma_1 \cup \sigma_2$ est bien définie : elle agit comme σ_1 sur les variables de C_1 et comme σ_2 sur celles de C_2.

Soit a' l'atome clos sur lequel on coupe : a' est positif dans C_1' et négatif dans C_2'. On pose

$$P_1' = \{b \in P_1 \mid \sigma_1(b) = a'\}, \qquad N_2' = \{b \in N_2 \mid \sigma_2(b) = a'\}.$$

Ces ensembles sont non vides, et τ unifie $P_1' \cup N_2'$. Soit σ un mgu de $P_1' \cup N_2'$: il existe β tel que $\tau = \beta \circ \sigma$. □

Lemme du relèvement (suite)

Suite de la preuve.

Soit C le résolvant de C_1, C_2 pour P_1', N_2' et σ :
$C = \big(\sigma(N_1 \cup (N_2 \setminus N_2')),\ \sigma((P_1 \setminus P_1') \cup P_2)\big)$. Couper sur a' retire exactement les images de P_1' et de N_2', donc

$$C' = \big(\tau(N_1 \cup (N_2 \setminus N_2')),\ \tau((P_1 \setminus P_1') \cup P_2)\big) = \beta(C).$$

□

Relèvement d'une réfutation close

Soit $C'_0, \ldots, C'_n, C'_{n+1}, \ldots, C'_m = \bot$ une réfutation close de $S_c = \{C'_0, \ldots, C'_n\}$. Par induction, avec le lemme du relèvement, on construit des clauses $C_0, \ldots, C_m$ et des substitutions σ_i telles que

- $C_0, \ldots, C_n \in S$ (à renommage près) et $C'_i = \sigma_i(C_i)$;
- chaque C_i, pour $i > n$, est un résolvant de clauses qui la précèdent.

$$\begin{array}{ccccccc} C_0, \ldots, C_n & \to & C_{n+1} & \to & \cdots & \to & C_m \quad \text{(premier ordre)} \\ \downarrow & & \downarrow & & & & \downarrow \\ C'_0, \ldots, C'_n & \to & C'_{n+1} & \to & \cdots & \to & \bot \quad \text{(clos)} \end{array}$$

Comme $\sigma_m(C_m) = \bot$, on a nécessairement $C_m = \bot$: c'est une réfutation de S.

Exemple 105

La coupure close entre $p(a)$ et $\neg p(a) \vee q(f(a))$ donne $q(f(a))$. Au premier ordre, la résolution entre $p(x)$ et $\neg p(y) \vee q(f(y))$, avec le mgu (y/x), donne $q(f(y))$, dont $q(f(a))$ est une instance.

Terminaison ? Non !

- En logique propositionnelle, saturer termine toujours.
- Au premier ordre, la saturation peut ne pas terminer, notamment lorsque S est satisfaisable.

Exemple 106 (saturation infinie)

Soit $S = \{r(x, f(x)),\ \neg r(x, y) \vee \neg r(y, z) \vee r(x, z),\ \neg r(x, x)\}$: r est transitive et irréflexive, et chaque élément est en relation avec son image par f.

- La résolution produit $r(x, f(f(x)))$, puis $r(x, f(f(f(x))))$, et ainsi de suite, sans jamais atteindre $\bot$.
- L'ensemble est en effet satisfaisable, par exemple dans $(\mathbb{N}, <, succ)$.

Bilan

La résolution est une semi-procédure : si S est insatisfaisable, on finit par trouver $\bot$. Mieux est impossible, puisque la satisfaisabilité au premier ordre est indécidable (Church, 1936).

Cas particulier : clauses sans symbole de fonction

Soit S un ensemble fini de clauses dont le langage ne contient que des constantes (au moins une, sinon on en ajoute une), réunies dans l'ensemble C.

- $\mathcal{H}_D = C$ est fini, donc $\mathcal{H}_B = \{p(c_1, \ldots, c_n) \mid p|_n \in \mathcal{L},\ c_i \in C\}$ l'est aussi : $|\mathcal{H}_B| = \sum_{p|_n} |C|^n$.
- L'ensemble $Gr(S)$ des instances closes des clauses de S est donc fini.

SatSansFonction(S)

1. Calculer $Gr(S)$: instancier chaque clause par toutes les substitutions $Var \to C$.
2. Voir chaque atome clos comme une proposition et décider la satisfaisabilité de $Gr(S)$ par la résolution propositionnelle (ou DPLL).
3. **retourner** la réponse.

Théorème 107

S est satisfaisable ssi $Gr(S)$ l'est. L'algorithme est donc correct et complet : la satisfaisabilité est décidable pour ce fragment.

- Preuve : S a un modèle ssi il a un modèle de Herbrand, c'est-à-dire une valuation de $\mathcal{H}_B$ qui rend vraies toutes les clauses de $Gr(S)$.
- Formules concernées : celles dont la forme prénexe a un préfixe $\exists^* \forall^*$, car un $\exists$ placé *après* un $\forall$ produit une fonction de Skolem.
- Coût : $|Gr(S)|$ est exponentiel en le nombre de variables par clause.

Sans fonction : une perte de généralité

Idée naturelle : remplacer $f|_1$ par un prédicat $F|_2$, où $F(x, y)$ signifie $y = f(x)$. Il faut alors dire que F est le graphe d'une fonction :

fonctionnelle (au plus une image)	$\forall x \forall y \forall z\, (F(x, y) \wedge F(x, z) \to y = z)$	sans fonction
totale (au moins une image)	$\forall x\, \exists y\, F(x, y)$	$\rightsquigarrow \quad F(x, g(x))$!

La skolémisation de la totalité réintroduit un symbole de fonction. Sans cet axiome, F n'est qu'une fonction *partielle*.

Exemple 108 (un ensemble qui n'a que des modèles infinis)

$S = \{\neg r(x, x), \quad \neg r(x, y) \vee \neg r(y, z) \vee r(x, z), \quad r(x, f(x))\}$: r est un ordre strict sans élément maximal. S est satisfaisable dans $(\mathbb{N}, <, succ)$, mais tous ses modèles sont infinis. Avec F et sans la totalité, la structure à un élément où r et F sont vides est un modèle : on a perdu « chaque élément a un successeur ».

Pourquoi on ne peut pas faire mieux

- Un ensemble *satisfaisable* de clauses sans fonction a un modèle de Herbrand, donc un modèle fini (de taille $\leq |C|$). Aucun ensemble de ce type n'est donc équivalent à S.
- Si tout se ramenait au cas sans fonction, la satisfaisabilité au premier ordre serait décidable, ce qui contredit le théorème de Church.

Pièges fréquents

- **Priorités** : $\exists x\, p(x) \wedge q(x)$ se lit $(\exists x\, p(x)) \wedge q(x)$.
- **Capture** : on ne substitue t à x que si t est libre pour x, en renommant d'abord les variables liées si nécessaire.
- **Variables fraîches** : pour $\forall_i$ et $\exists_e$, x_0 n'apparaît ni hors de la boîte ni dans χ.
- $\forall\exists \neq \exists\forall$: $\exists x \forall y \models \forall y \exists x$, mais pas l'inverse.
- **Antécédent d'une implication** : un quantificateur qui en sort change de nature.
- **Skolémisation** : la fonction de Skolem dépend de *tous* les $\forall$ qui précèdent le $\exists$, et seulement d'eux. Le symbole est toujours nouveau, et l'équivalence n'est pas préservée.
- **Nier la conclusion** avant de skolémiser.
- **Une seule coupure à la fois** en logique propositionnelle.
- **Séparer les clauses** avant de les résoudre.
- **Test d'occurrence** : x ne s'unifie jamais avec un terme qui contient x strictement.